SECURE USER APPLICATION USING .NET

Name: Diksha Chaudhary

Course: .NET Full Stack Development

Assignment: W2 Day 5 Daily Coding Assignment

Technology Used: C#, .NET Class Library, MSTest

---

1. Objective

The main objective of this assignment is to implement a secure user management system that demonstrates authentication, encryption, exception handling, logging, and unit testing using .NET. The application ensures that sensitive information such as passwords is stored securely and that user data can be safely encrypted and decrypted.

---

2. Introduction

Security is an important aspect of modern application development. This assignment focuses on implementing core security features such as password hashing, data encryption using AES, structured error handling, and logging. Unit testing is also performed to verify that each module works correctly.

The application is developed using a class library for the core functionality and an MSTest project for testing.

---

3. Project Structure

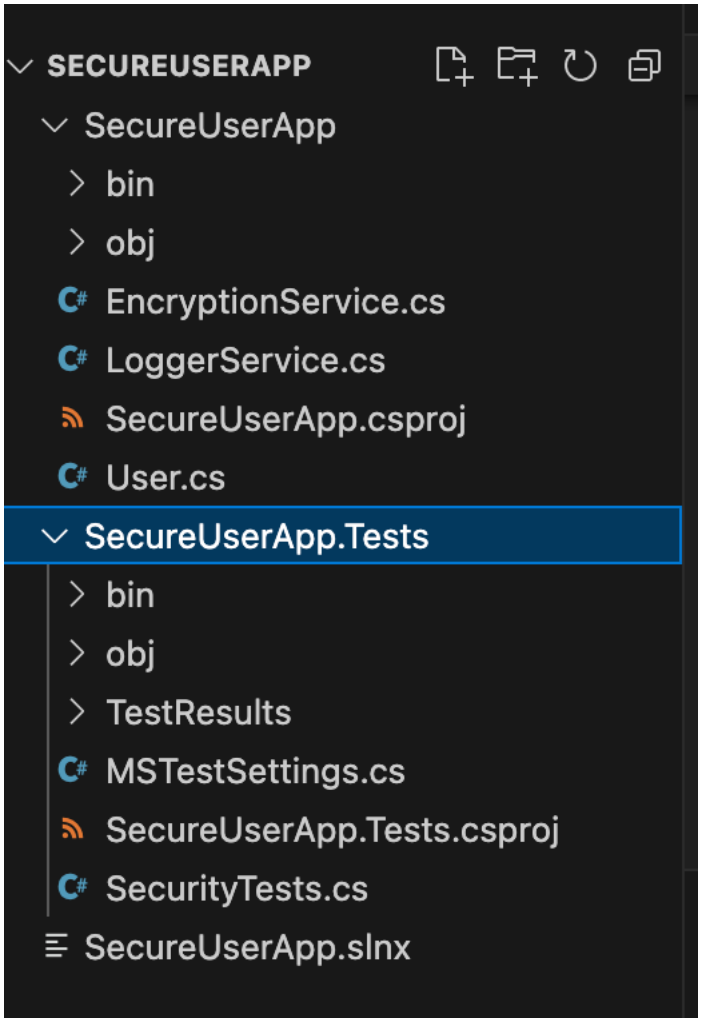
The solution contains two projects:

SecureUserApp

This project contains the business logic for user authentication, encryption, and logging.

SecureUserApp.Tests

This project contains unit tests to validate the functionality of the application.



---

4. User Authentication

A User class is implemented to handle user registration and authentication.

During registration, the password is not stored in plain text. Instead, it is converted into a hashed format using the SHA256 algorithm. This ensures that even if the data is accessed, the original password cannot be retrieved.

During login, the entered password is hashed again and compared with the stored hash. If both values match, the user is authenticated successfully.

This approach improves security and follows standard authentication practices.

User.cs | EncryptionService.cs | SecurityTests.cs | LoggerService.cs

SecureUserApp > User.cs > User > HashedPassword

```
using System;
using System.Security.Cryptography;
using System.Text;

namespace SecureUserApp
{
    4 references
    public class User
    {
        1 reference
        public string Username { get; set; }
        3 references
        public string HashedPassword { get; set; }
        3 references
        public User(string username, string password)
        {
            Username = username;
            HashedPassword = HashPassword(password);
        }
        2 references
        private string HashPassword(string password)
        {
            using (SHA256 sha256 = SHA256.Create())
            {
                byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));

                StringBuilder builder = new StringBuilder();

                foreach (var b in bytes)
                    builder.Append(b.ToString("x2"));

                return builder.ToString();
            }
        }
        2 references
        public bool Authenticate(string password)
        {
            string hashedInput = HashPassword(password);
            return HashedPassword == hashedInput;
        }
    }
}
```

---

5. Password Hashing using SHA256

SHA256 hashing is used to convert the password into a fixed-length encrypted value.

Key points:

The same input always produces the same hash

The original password cannot be retrieved from the hash

Plain text passwords are never stored

This ensures that the application follows secure password storage techniques.

---

6. Data Encryption and Decryption

An EncryptionService class is implemented to protect sensitive data.

AES (Advanced Encryption Standard) is used for encryption and decryption.

Encryption process:

The original text is converted into encrypted format

The output is stored as a Base64 string

Decryption process:

The encrypted text is converted back to its original readable format

This ensures that confidential data remains secure during storage or transmission.

User.cs | EncryptionService.cs | SecurityTests.cs | LoggerService.cs

SecureUserApp > EncryptionService.cs > EncryptionService > Encrypt

```
using System;
using System.IO;
using System.Security.Cryptography;
using System.Text;
using Serilog;

namespace SecureUserApp
{
    2 references
    public class EncryptionService
    {
        2 references
        private readonly byte[] key = Encoding.UTF8.GetBytes("1234567890123456");
        2 references
        private readonly byte[] iv = Encoding.UTF8.GetBytes("6543210987654321");
        1 reference
        public string Encrypt(string plainText)
        {
            try
            {
                using Aes aes = Aes.Create();
                aes.Key = key;
                aes.IV = iv;
                ICryptoTransform encryptor = aes.CreateEncryptor();
                using MemoryStream ms = new MemoryStream();
                using CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write);
                using StreamWriter sw = new StreamWriter(cs);

                sw.Write(plainText);
                sw.Flush();
                cs.FlushFinalBlock();

                return Convert.ToBase64String(ms.ToArray());
            }
            catch (Exception ex)
            {
                Log.Error(ex, "Encryption failed");
                throw;
            }
        }

        2 references
        public string Decrypt(string cipherText)
        {
            try
            {
                using Aes aes = Aes.Create();
                aes.Key = key;
                aes.IV = iv;

                ICryptoTransform decryptor = aes.CreateDecryptor();

                using MemoryStream ms = new MemoryStream(Convert.FromBase64String(cipherText));
                using CryptoStream cs = new CryptoStream(ms, decryptor, CryptoStreamMode.Read);
                using StreamReader sr = new StreamReader(cs);

                return sr.ReadToEnd();
            }
            catch (Exception ex)
            {
                Log.Error(ex, "Decryption failed");
                throw;
            }
        }
    }
}
```

7. Exception Handling

Exception handling is implemented using try–catch blocks.

If an error occurs during encryption or decryption:

The exception is caught

The error is logged

The exception is re-thrown for testing

This prevents the application from crashing and makes debugging easier.

---

8. Logging using Serilog

Serilog is used to log application errors into a text file.

Whenever an exception occurs, the following details are stored:

Error message

Stack trace

Time of occurrence

Logging helps in tracking issues and maintaining the application efficiently.

User.cs | EncryptionService.cs | SecurityTests.cs | LoggerService.cs

SecureUserApp > LoggerService.cs > ...

```
using Serilog;

namespace SecureUserApp
{
    0 references
    public static class LoggerService
    {
        0 references
        public static void ConfigureLogger()
        {
            Log.Logger = new LoggerConfiguration()
                .WriteTo.File("logs.txt", rollingInterval: RollingInterval.Day)
                .CreateLogger();
        }
    }
}
```

---

9. Unit Testing

Unit testing is performed using MSTest to verify the correctness of each feature.

The following test cases are implemented:

User registration hashes the password

Authentication succeeds with the correct password

Authentication fails with an incorrect password

Encryption and decryption return the original text

Invalid decryption input throws an exception

All test cases are executed successfully.

User.cs | EncryptionService.cs | SecurityTests.cs | LoggerService.cs

SecureUserApp.Tests > SecurityTests.cs > SecurityTests > Encrypt_Decrypt_Should_Return_Original_Text

```
1   using Microsoft.VisualStudio.TestTools.UnitTesting;
2   using SecureUserApp;
3   using System;
4
5   namespace SecureUserApp.Tests
6   {
7       [TestClass]
        0 references
8       public class SecurityTests
9       {
10          [TestMethod]
            0 references
11          public void Register_User_Should_Hash_Password()
12          {
13              var user = new User("diksha", "1234");
14              Assert.AreNotEqual("1234", user.HashedPassword);
15          }
16          [TestMethod]
            0 references
17          public void Authenticate_With_Correct_Password_Should_Return_True()
18          {
19              var user = new User("diksha", "1234");
20              var result = user.Authenticate("1234");
21              Assert.IsTrue(result);
22          }
23          [TestMethod]
            0 references
24          public void Authenticate_With_Wrong_Password_Should_Return_False()
25          {
26              var user = new User("diksha", "1234");
27              var result = user.Authenticate("wrong");
28              Assert.IsFalse(result);
29          }
30          [TestMethod]
            0 references
31          public void Encrypt_Decrypt_Should_Return_Original_Text()
32          {
33              var service = new EncryptionService();
34              string original = "Hello";
35              string encrypted = service.Encrypt(original);
36              string decrypted = service.Decrypt(encrypted);
37              Assert.AreEqual(original, decrypted);
38          }
39          [TestMethod]
            0 references
40          public void Decrypt_Invalid_Data_Should_Throw_Exception()
41          {
42              var service = new EncryptionService();
43              try
44              {
45                  service.Decrypt("invalid_cipher");
46                  Assert.Fail("Expected FormatException was not thrown.");
47              }
48              catch (FormatException)
49              {
50                  return;
51              }
52              Assert.Fail("Expected FormatException was not thrown.");
53          }
54      }
55  }
```

10. Test Results

All unit tests passed successfully, which confirms that:

The authentication system works correctly

The encryption and decryption process is accurate

Exception handling is functioning as expected

This ensures the reliability of the application.

```
● (base) dikshachaudhary@Dikshas-MacBook-Air SecureUserApp.Tests % dotnet test
  Restore complete (0.3s)
    SecureUserApp net10.0 succeeded (3.2s) → /Users/dikshachaudhary/SecureUserApp/SecureUserApp/bin/Debug/net10.0/SecureUserApp.dll
    SecureUserApp.Tests net10.0 succeeded (0.6s) → bin/Debug/net10.0/SecureUserApp.Tests.dll
  Test Parallelization enabled for /Users/dikshachaudhary/SecureUserApp/SecureUserApp.Tests/bin/Debug/net10.0/SecureUserApp.Tests.dll (Workers: 8, Scope: MethodLevel)
    SecureUserApp.Tests test net10.0 succeeded (1.2s)

  Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 1.1s
  Build succeeded in 5.8s
```

---

11. Conclusion

This assignment helped in understanding how security features are implemented in a .NET application. Password hashing, AES encryption, structured exception handling, logging, and unit testing were successfully implemented.

The application demonstrates how sensitive data can be protected and how testing ensures the correctness of functionality.